Grado en Inteligencia Artificial · Universidade da Coruña · Curso 2026/27

Práctica 2. Indicaciones técnicas

NEAT-Python sobre el entorno de la Práctica 1

Integración con Gymnasium y ROS 2, archivo de configuración, función de calidad, presupuesto de cómputo, puntos de control, registro y figuras

Complementa a	Enunciado de la Práctica 2
Punto de partida	El entorno de Gymnasium de la Práctica 1 funcionando contra RoboboSim, con el evaluador del anexo B
Código	Los fragmentos de este documento están probados con NEAT-Python 2.0, la versión instalada en la imagen. Son esqueletos: los nombres del entorno y de la función de calidad son de cada grupo

1. Qué hay en el contenedor

- **NEAT-Python 2.0** está en la nueva versión de la imagen. Buena parte de la documentación en línea y de los ejemplos de internet corresponden a versiones anteriores, con diferencias en la configuración y en los puntos de control: en caso de duda, vale lo que dice este documento.
- **Matplotlib** viene como dependencia de Stable-Baselines3: es suficiente para todas las figuras.

1.1. Actualizar la imagen antes de empezar

La imagen construida para la Práctica 1 no incluye NEAT-Python. Hay que descargar la versión actual de la carpeta `pc-robobo` y reconstruir la imagen rehaciendo sus capas. Sin opciones, Docker reutiliza las capas en caché y no instala nada nuevo. Desde la carpeta del sistema operativo correspondiente:

```
./build.sh --all          # macOS y Ubuntu
.\build.ps1 -All          # Windows, desde PowerShell
```

La reconstrucción tarda varios minutos, porque vuelve a instalar los paquetes de Python y a compilar `robobo_ros2`. Después hay que detener el contenedor que estuviera en marcha (`docker compose down`), arrancarlo de nuevo con `start` y comprobar desde una terminal del contenedor que la versión es la correcta:

```
pip show neat-python      # debe indicar Version: 2.0.0
```

El código de la carpeta `docker-ros/ws` no se pierde al reconstruir, porque está en el equipo anfitrión.

2. Cómo encaja NEAT en el entorno de la Práctica 1

En la Práctica 1, Stable-Baselines3 llamaba a `reset()` y `step()` y aprendía de la recompensa de cada paso. Ahora el bucle lo escribe cada grupo y la evolución sólo recibe un número por individuo. El esquema es este:

1. `Population.run(eval_genomes, n)` entrega a `eval_genomes` la lista de genomas de la generación.
2. Para cada genoma se construye su red, que controla al seguidor durante uno o varios episodios del entorno de la Práctica 1.
3. Con lo registrado en esos episodios se calcula la calidad (*fitness*) y se asigna a `genoma.fitness`.
4. NEAT-Python agrupa en especies, selecciona, cruza y muta, y genera la población siguiente.

El entorno no necesita cambios para esto: sigue siendo un `gym.Env`. Lo que sí conviene es que `step()` devuelva en `info` las magnitudes privilegiadas que necesite la calidad (separación real, causa de fin, posición respecto al carril), que ya se calculaban en la Práctica 1 para las métricas. La recompensa que devuelve `step()` se puede ignorar.

```
import numpy as np
import neat
from p1_entorno import SeguidorEnv          # el entorno de la Práctica 1

env = SeguidorEnv()                         # un único entorno y un único nodo
PASOS_EVAL = 150                            # 30 s con pasos de 0,2 s
CONTADOR = {"pasos": 0, "evaluaciones": 0}

def evaluar_genoma(genoma, config):
    red = neat.nn.FeedForwardNetwork.create(genoma, config)
    obs, info = env.reset()
    registro = []                       # lo que necesite la calidad
    for _ in range(PASOS_EVAL):
        accion = np.array(red.activate(obs), dtype=np.float32)
        obs, _, terminado, truncado, info = env.step(accion)
        registro.append(info)
        if terminado or truncado:
            break
    CONTADOR["pasos"] += len(registro)
    CONTADOR["evaluaciones"] += 1
    return calidad(registro)                # la define cada grupo (apartado 4)

def eval_genomes(genomas, config):
    for _, genoma in genomas:
        genoma.fitness = evaluar_genoma(genoma, config)
```

- **Un único entorno**, creado una vez fuera de la función de evaluación. Crear uno por genoma crearía un nodo de ROS 2 por genoma.
- **La observación** se pasa tal cual la normaliza el entorno. NEAT inicializa los pesos con media 0 y desviación 1, así que las entradas deben estar en rangos comparables, igual que en la Práctica 1.
- **La salida de la red**: `activate()` devuelve una lista con `num_outputs` valores. Si el espacio de acción es `Box(-1, 1)`, la activación de salida debe dar valores en ese rango (apartado 3).
- **Varios episodios por individuo**: la calidad se agrega con la media o con el mínimo. El mínimo favorece a los individuos robustos, pero penaliza mucho un episodio desafortunado.

El programa principal crea la población, le añade los registros y lanza la evolución. Los registros (*reporters* en NEAT-Python, de ahí `add_reporter`) son objetos a los que NEAT avisa en cada etapa de la evolución: al empezar una generación, tras evaluarla y al terminarla. Unos los trae la biblioteca (salida por consola, estadísticas, puntos de control) y otros los escribe cada grupo:

```
import os, pickle
from registro import RegistroGeneracion       # apartado 6

os.makedirs("chk", exist_ok=True)       # el Checkpointer no crea carpetas
config = neat.Config(neat.DefaultGenome, neat.DefaultReproduction,
                     neat.DefaultSpeciesSet, neat.DefaultStagnation,
                     "config-p2.ini")
poblacion = neat.Population(config)
poblacion.add_reporter(neat.StdOutReporter(True))
estadisticas = neat.StatisticsReporter()
poblacion.add_reporter(estadisticas)
poblacion.add_reporter(neat.Checkpointer(generation_interval=1,
                                         filename_prefix="chk/p2-"))
poblacion.add_reporter(RegistroGeneracion("evolucion.csv", "genomas"))

mejor = poblacion.run(eval_genomes, 30)       # 30 generaciones
with open("mejor_genoma.pkl", "wb") as f:
    pickle.dump(mejor, f)
```

`run()` devuelve el genoma con la mayor calidad registrada en toda la evolución. Con una calidad ruidosa puede ser un individuo afortunado: antes de elegir el controlador final, conviene volver a evaluar con varios episodios a unos pocos candidatos, por ejemplo `estadisticas.best_unique_genomes(5)` o los mejores genomas guardados en las últimas generaciones.

3. El archivo de configuración

NEAT-Python lee todos sus parámetros de un archivo de texto. Éste es un punto de partida razonable para este problema; `num_inputs` y `num_outputs` deben coincidir con la dimensión de la observación y de la acción del entorno de cada grupo.

```
[NEAT]
fitness_criterion      = max
fitness_threshold      = 1000.0
no_fitness_termination = True
pop_size               = 20
reset_on_extinction    = True
seed                   = 42

[DefaultGenome]
num_inputs         = 5
num_hidden         = 0
num_outputs        = 2
feed_forward       = True
initial_connection = full_direct

activation_default      = tanh
activation_mutate_rate  = 0.0
activation_options      = tanh
aggregation_default     = sum
aggregation_mutate_rate = 0.0
aggregation_options     = sum

bias_init_mean      = 0.0
bias_init_stdev     = 1.0
bias_max_value      = 5.0
bias_min_value      = -5.0
bias_mutate_power   = 0.5
bias_mutate_rate    = 0.7
bias_replace_rate   = 0.1

response_init_mean    = 1.0
response_init_stdev   = 0.0
response_max_value    = 5.0
response_min_value    = -5.0
response_mutate_power = 0.0
response_mutate_rate  = 0.0
response_replace_rate = 0.0

weight_init_mean    = 0.0
weight_init_stdev   = 1.0
weight_max_value    = 5.0
weight_min_value    = -5.0
weight_mutate_power = 0.5
weight_mutate_rate  = 0.8
weight_replace_rate = 0.1

enabled_default     = True
enabled_mutate_rate = 0.01

conn_add_prob    = 0.3
conn_delete_prob = 0.1
node_add_prob    = 0.1
node_delete_prob = 0.05
```

```
compatibility_disjoint_coefficient = 1.0
compatibility_weight_coefficient   = 0.5

[DefaultSpeciesSet]
compatibility_threshold = 3.0

[DefaultStagnation]
species_fitness_func = max
max_stagnation       = 8
species_elitism      = 1

[DefaultReproduction]
elitism            = 2
survival_threshold = 0.3
min_species_size   = 2
```

Todos los parámetros deben estar presentes, salvo `seed`, que es opcional: NEAT-Python no admite que falte ninguno ni que sobre alguno desconocido. `seed` fija la semilla de la evolución (no la del simulador), de modo que dos ejecuciones con la misma semilla generan las mismas mutaciones y cruces. Los parámetros que más influyen en esta práctica son:

Parámetro	Qué controla	Indicación para esta práctica
pop_size	Individuos por generación	Entre 15 y 30. El coste de cada generación es proporcional
no_fitness_termination	Si la evolución se detiene al alcanzar fitness_threshold	True: con una calidad ruidosa es mejor parar por número de generaciones
num_inputs, num_outputs	Tamaño de la red	Igual a la dimensión de la observación y de la acción
feed_forward	Tipo de red	True: red prealimentada, sin ciclos
initial_connection	Conexiones de la población inicial	full_direct: todas las entradas a todas las salidas. Un controlador reactivo ya es expresable desde la primera generación. Con unconnected se pierden generaciones
activation_options	Funciones de activación disponibles	tanh si la acción está en [−1, 1]. sigmoid da salidas en [0, 1]
weight_*, bias_*	Inicialización y mutación de pesos y sesgos	Los valores de arriba. weight_mutate_power regula el tamaño de cada mutación
conn_add_prob, node_add_prob	Velocidad de crecimiento de la topología	Bajas al principio: la tarea no requiere redes grandes
compatibility_threshold	Umbral de distancia para pertenecer a una especie	Ajustarlo para tener entre 3 y 6 especies. Más alto, menos especies
max_stagnation	Generaciones sin mejorar antes de eliminar una especie	Entre 8 y 15, según el número total de generaciones
elitism, species_elitism	Individuos y especies que se conservan intactos	2 y 1. Los individuos conservados se vuelven a evaluar en cada generación
survival_threshold	Fracción de cada especie que puede reproducirse	Entre 0,2 y 0,3

Las activaciones de NEAT-Python están escaladas. `tanh` calcula tanh(2,5·z) y `sigmoid`, 1/(1+e^(−5·z)): saturan mucho antes que las funciones habituales. Con pesos del orden de 1, la salida pasa con facilidad de un extremo a otro. Es una de las razones por las que la normalización de las entradas importa.

Se recomienda copiar el archivo de configuración junto a los resultados de cada evolución: es parte de las evidencias y la única forma de saber después con qué parámetros se obtuvo cada genoma.

4. La función de calidad

El enunciado deja el diseño de la función de calidad a cada grupo. Estas son indicaciones sobre cómo se comporta NEAT-Python con ella; no son una fórmula:

- **Escala.** NEAT-Python normaliza la calidad de cada especie dividiendo por el rango de la población, pero ese rango nunca es menor que 1. Si todas las calidades caen en [0, 1], las diferencias se aplastan y la presión selectiva casi desaparece. Conviene trabajar en una escala como 0–100.
- **Siempre un número.** Todos los genomas deben recibir una calidad real y finita. Si la evaluación lanza una excepción y un genoma se queda con `None`, la evolución se detiene con un error al final de la generación.
- **Normalizada por la duración máxima.** Si la calidad se divide por el número máximo de pasos del episodio, sigue siendo comparable aunque la duración cambie entre etapas de la evolución.
- **Con gradiente desde la primera generación.** Al principio casi todos los individuos fallan en pocos segundos. La calidad tiene que distinguir al que pierde al guía en el segundo 2 del que lo pierde en el segundo 8; si no, la selección es aleatoria.
- **Terminación y penalización.** Si un fallo termina el episodio, los pasos no vividos ya no suman: eso es una penalización implícita. Añadir otra explícita es una decisión que hay que justificar, porque cambia el equilibrio entre arriesgar y no arriesgar.
- **Comprobar con un controlador conocido.** El controlador programado a mano en el reto, evaluado con la misma función de calidad, da una referencia de qué valores son buenos antes de lanzar ninguna evolución.

5. Presupuesto de cómputo

Cada evaluación transcurre en tiempo real, así que el coste de una evolución se puede acotar antes de lanzarla. Con P individuos, k episodios por individuo, una duración máxima de episodio T y un tiempo de reinicio de la simulación T_r:

> **Duración máxima de una generación ≈ P × k × (T_r + T)**, y de la evolución completa, esa cantidad multiplicada por el número de generaciones G.

La tabla da algunos órdenes de magnitud con T_r = 3 s, que cada grupo debe sustituir por el valor que mida. Son cotas superiores: en las primeras generaciones muchos episodios terminan antes, pero las últimas se acercan al máximo, porque los individuos buenos agotan el episodio.

P	k	T (s)	Generación (máx.)	G	Evolución (máx.)
15	1	30	8 min	40	5,5 h
20	1	20	8 min	30	3,8 h
20	1	30	11 min	30	5,5 h
30	1	30	16,5 min	30	8,2 h
20	2	30	22 min	30	11 h

Algunas formas de ajustar el coste:

- **Medir antes de estimar.** Lanzar dos o tres generaciones, mirar la columna de segundos del registro (apartado 6) y medir el tiempo de `reset()`. Con eso se decide el resto.

- **Episodios de evolución cortos.** El episodio de evaluación del anexo B dura 120 s, pero para evolucionar pueden bastar 20–40 s, siempre que incluyan al menos una curva y, en C4, alguna parada.
- **La misma configuración del guía para toda una generación.** Así todos los individuos se comparan en igualdad de condiciones. Si se cambia de una generación a otra (por ejemplo, alternando C2, C3 y C4), se evoluciona un único controlador para varias configuraciones.
- **Evolución incremental.** Se evoluciona con una configuración sencilla y, desde el último punto de control, se continúa con otra más difícil. La configuración del guía y la duración del episodio son parámetros del entorno, no de NEAT, así que se pueden cambiar al reanudar.
- **Dos ordenadores, dos evoluciones.** Cada pareja tiene dos equipos: se pueden lanzar a la vez dos semillas o dos configuraciones. `neat.ParallelEvaluator` no sirve aquí, porque cada simulador controla un único seguidor.
- **El multiplicador de velocidad del simulador** reduce el tiempo de reloj del episodio, pero cambia la dinámica del problema, como se vio en la Práctica 1. Debe ser el mismo en la evolución y en la evaluación.

6. Registro, puntos de control y reanudación

Una evolución de varias horas se puede interrumpir: el nodo puente pierde la conexión con el simulador y la evolución se queda esperando sin dar error, como ya ocurría en la Práctica 1. Hay que poder reanudarla sin perder lo hecho.

6.1. Puntos de control

`neat.Checkpointer(generation_interval=1)` guarda en cada generación un archivo con la población, las especies y el estado del generador aleatorio. Para reanudar:

```
poblacion = neat.Checkpointer.restore_checkpoint("chk/p2-12")
# los registros no se recuperan: se vuelven a añadir, igual que al empezar
poblacion.add_reporter(neat.StdOutReporter(True))
poblacion.add_reporter(neat.Checkpointer(generation_interval=1,
                                         filename_prefix="chk/p2-"))
poblacion.add_reporter(RegistroGeneracion("evolucion.csv", "genomas"))
mejor = poblacion.run(eval_genomes, 18)       # las generaciones que faltan
```

- El archivo `p2-N` contiene la población lista para empezar la generación N: se usa el de número más alto.
- El punto de control guarda también la configuración de NEAT con la que se creó: cambiar el archivo de configuración no afecta a una evolución reanudada, salvo que se pase la nueva configuración con el argumento `new_config` de `restore_checkpoint`.
- **El Checkpointer serializa los registros con pickle.** Un registro propio que guarde un archivo abierto, el entorno o el nodo de ROS 2 hace fallar el guardado con `cannot pickle`. El registro de abajo abre el archivo en cada escritura por eso.
- Si la comunicación se cuelga a mitad de una generación, no hay que asignar una calidad de 0 al genoma afectado: se estaría castigando a un individuo por un fallo de la infraestructura. Es mejor detener la evolución, relanzar el nodo puente y reanudar desde el último punto de control.

6.2. Registro por generación

`StatisticsReporter` guarda las estadísticas en memoria, pero se pierden si la evolución se interrumpe y empiezan vacías al reanudar. Un registro propio que escriba una fila por generación en un CSV es más robusto y proporciona los datos de todas las figuras. Éste guarda además el mejor genoma de cada generación y continúa los contadores si se reanuda:

```
import csv, os, pickle, statistics, time
import neat
```

```
CONTADOR = {"pasos": 0, "evaluaciones": 0}   # lo actualiza evaluar_genoma()

class RegistroGeneracion(neat.reporting.BaseReporter):
    CAMPOS = ["generacion", "cal_mejor", "cal_media", "cal_desv", "especies",
              "nodos_mejor", "conexiones_mejor", "evaluaciones", "pasos",
              "segundos"]

    def __init__(self, ruta_csv, carpeta_genomas):
        self.ruta, self.carpeta = ruta_csv, carpeta_genomas
        os.makedirs(carpeta_genomas, exist_ok=True)
        self.base = {"evaluaciones": 0, "pasos": 0, "segundos": 0.0}
        if os.path.exists(ruta_csv):              # se reanuda una evolución
            with open(ruta_csv) as f:
                filas = list(csv.DictReader(f))
            if filas:
                self.base = {k: float(filas[-1][k]) for k in self.base}
        else:
            with open(ruta_csv, "w", newline="") as f:
                csv.writer(f).writerow(self.CAMPOS)
        self.inicio = dict(CONTADOR)              # por si el proceso ya había evaluado
        self.t0, self.gen = time.time(), 0

    def start_generation(self, generation):
        self.gen = generation

    def post_evaluate(self, config, population, species, best_genome):
        cal = [g.fitness for g in population.values()]
        nodos, conexiones = best_genome.size()
        fila = [self.gen, best_genome.fitness,
                statistics.mean(cal), statistics.pstdev(cal),
                len(species.species), nodos, conexiones,
                int(self.base["evaluaciones"] + CONTADOR["evaluaciones"]
                    - self.inicio["evaluaciones"]),
                int(self.base["pasos"] + CONTADOR["pasos"] - self.inicio["pasos"]),
                round(self.base["segundos"] + time.time() - self.t0, 1)]
        with open(self.ruta, "a", newline="") as f:
            csv.writer(f).writerow(fila)
        ruta = os.path.join(self.carpeta, f"mejor_gen{self.gen:03d}.pkl")
        with open(ruta, "wb") as f:
            pickle.dump(best_genome, f)
```

`best_genome.size()` devuelve el número de nodos (sin contar las entradas) y de conexiones activas: es la medida de complejidad que pide el enunciado. Las columnas de pasos y de segundos dan el coste que hay que declarar en la memoria.

7. Figuras

Todas las figuras se generan con código propio a partir del CSV de cada evolución. Para varias semillas se leen varios CSV, se alinean por generación y se dibuja la media con una banda (por ejemplo, con `fill_between`).

- **Curva de evolución**: `cal_mejor` y `cal_media` frente a `generacion`, con `cal_desv` como banda de la media.
- **Especiación**: `StatisticsReporter.get_species_sizes()` devuelve, para cada generación, el tamaño de todas las especies que han existido (0 si aún no existían o ya se extinguieron). Si la evolución se reanudó, sólo contiene las generaciones posteriores; para la figura completa, se guarda en cada sesión con `estadisticas.save()` o con un registro propio.

```
tam = np.array(estadisticas.get_species_sizes()).T    # especies × generaciones
plt.stackplot(range(tam.shape[1]), tam)
plt.xlabel("Generación"); plt.ylabel("Individuos por especie")
```

- **Topología de la red**: la función siguiente dibuja una red por capas con Matplotlib, sin Graphviz. El grosor de cada flecha es proporcional al peso y el color indica su signo.

```
import matplotlib.pyplot as plt

def dibujar_red(genoma, config, nombres_entrada, nombres_salida, ruta):
    gc = config.genome_config
    entradas, salidas = list(gc.input_keys), list(gc.output_keys)
    activas = [c.key for c in genoma.connections.values() if c.enabled]
    # Profundidad de cada nodo: camino más largo desde las entradas
    prof = {n: 0 for n in entradas}
    for _ in range(len(genoma.nodes) + 1):
        for (a, b) in activas:
            nueva = prof[a] + 1 if a in prof and a != b else -1
            if prof.get(b, -1) < nueva <= len(genoma.nodes):
                prof[b] = nueva
    ocultas = [n for n in prof if n not in entradas and n not in salidas]
    capas = [entradas]
    for p in sorted({prof[n] for n in ocultas}):
        capas.append(sorted(n for n in ocultas if prof[n] == p))
    capas.append(salidas)
    pos = {n: (i, j - (len(c) - 1) / 2)
           for i, c in enumerate(capas) for j, n in enumerate(c)}
    fig, ax = plt.subplots(figsize=(6, 4))
    for (a, b) in activas:
        if a in pos and b in pos:
            w = genoma.connections[(a, b)].weight
            ax.annotate("", xy=pos[b], xytext=pos[a], arrowprops=dict(
                arrowstyle="->", lw=0.5 + 1.5 * min(abs(w), 3) / 3,
                color="tab:green" if w > 0 else "tab:red"))
    nombres = dict(zip(entradas, nombres_entrada))
    nombres.update(zip(salidas, nombres_salida))
    for n, (x, y) in pos.items():
        ax.scatter(x, y, s=300, color="lightgrey", edgecolor="black", zorder=3)
        ax.text(x, y + 0.25, nombres.get(n, str(n)), ha="center", fontsize=8)
    ax.axis("off")
    fig.savefig(ruta, dpi=200, bbox_inches="tight")
```

Con los nombres de las entradas y de las salidas en la figura, la interpretación de la red que pide el enunciado se apoya directamente en ella.

8. Ejecutar y evaluar la red final

El evaluador del anexo B de la Práctica 1 se reutiliza. Si recibe el controlador como una función `observación → acción`, sirve igual para la red de NEAT y para la política de Stable-Baselines3, que es lo que necesita la comparación del apartado 6.5 del enunciado:

```
import pickle
import numpy as np
import neat
from stable_baselines3 import SAC

def controlador_neat(ruta_genoma, ruta_config):
    config = neat.Config(neat.DefaultGenome, neat.DefaultReproduction,
                         neat.DefaultSpeciesSet, neat.DefaultStagnation,
                         ruta_config)
    with open(ruta_genoma, "rb") as f:
        genoma = pickle.load(f)
    red = neat.nn.FeedForwardNetwork.create(genoma, config)
    return lambda obs: np.array(red.activate(obs), dtype=np.float32)

def controlador_sb3(ruta_modelo):
    modelo = SAC.load(ruta_modelo)                    # o PPO.load
    return lambda obs: modelo.predict(obs, deterministic=True)[0]

# red = controlador_neat("mejor_genoma.pkl", "config-p2.ini")
# evaluar_anexo_b(env, red, configuracion="C2")   # el evaluador de la P1
```

9. Errores frecuentes

Síntoma	Causa probable	Qué hacer
RuntimeError o TypeError al terminar una generación	Algún genoma se ha quedado sin calidad (genoma.fitness es None), normalmente por una excepción en la evaluación	Asignar siempre un valor real; capturar y registrar el error
Error al activar la red	num_inputs no coincide con la longitud de la observación	Igualarlos en el archivo de configuración
El robot nunca gira hacia un lado ni da marcha atrás	Salidas sigmoid en [0, 1] sobre una acción en [−1, 1]	Usar tanh o reescalar la salida
La calidad de toda la población es igual	La calidad no distingue entre individuos malos	Darle gradiente desde la primera generación (apartado 4)
La mejor calidad baja de una generación a otra	Calidad ruidosa: los individuos de élite se reevalúan	Es esperable. Se comenta en la memoria
cannot pickle al guardar el punto de control	Un registro propio guarda un archivo abierto, el entorno o el nodo de ROS 2	No guardar esos objetos en el registro
Una sola especie, o casi tantas como individuos	compatibility_threshold mal ajustado	Subirlo si hay demasiadas especies; bajarlo si hay una sola
CompleteExtinctionException	Todas las especies se han estancado a la vez	reset_on_extinction = True, o aumentar max_stagnation
La evolución se para sin mensaje	El nodo puente ha perdido la conexión con el simulador	Relanzar el puente y reanudar desde el último punto de control
Cambios en el archivo de configuración sin efecto	Se ha reanudado desde un punto de control	El punto de control guarda su propia configuración
«Best individual … meets fitness threshold» al terminar	Con no_fitness_termination = True, NEAT-Python lo imprime siempre al acabar las generaciones	Ignorarlo

10. Comprobaciones a realizar antes de lanzar una evolución larga

1. El entorno de la Práctica 1 funciona y la dimensión de su observación coincide con `num_inputs`.
2. Un genoma aleatorio completa un episodio de principio a fin y recibe una calidad.
3. El controlador del reto, evaluado con la misma función de calidad, da un valor razonable.
4. Dos generaciones completas; el punto de control se guarda y la evolución se reanuda desde él.
5. La duración de una generación está medida y el total, estimado.
6. La semilla (`seed`) está fijada y el archivo de configuración, copiado junto a los resultados.
7. La configuración del guía y el multiplicador de velocidad del simulador están anotados.
8. El CSV de la evolución se está escribiendo.